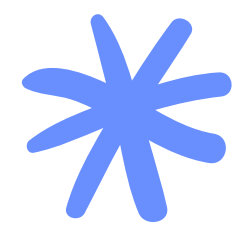


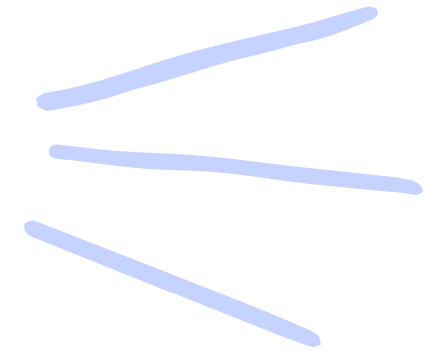


Systeme de Gestion de Base de Données





Chapitre 1: Introduction

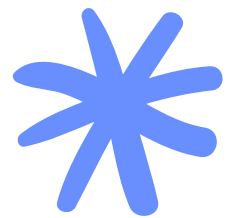


Les données dans notre vie quotidienne!

Chaque organisation utilise des informations dans son travail.

- * Une **bibliothèque** gère les **livres** et les **membres**.
 - * Une **entreprise** gère les **employés** et les **départements**.
 - * Une **université** gère les **étudiants**, les **matières** et les **notes**.
- Toutes ces informations sont appelées **données**.

Pour qu'elles soient utiles et faciles à exploiter, elles doivent être **structurées** et bien **organisées**.

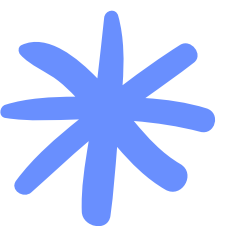


Avant les bases de données

Autrefois, les données étaient enregistrées dans des fichiers simples, comme des fichiers texte (créés avec le Bloc-notes) ou des fichiers Excel.

Mais cette méthode posait plusieurs problèmes :

- **Accès difficile** → il fallait écrire un programme pour lire ou modifier les fichiers.
- **Aucune sécurité** → tout le monde pouvait accéder ou changer les données.
- **Pas de travail en groupe** → impossible pour plusieurs personnes d'utiliser les fichiers en même temps.



La solution : les bases de données



Les bases de données ont été créées pour résoudre ces problèmes. Elles permettent de :

- Stocker les données de manière organisée et fiable,
- Partager facilement les informations entre plusieurs utilisateurs,
- Et protéger les données grâce à des droits d'accès et des contrôles de sécurité



Les Systèmes de Gestion de Bases de Données (SGBD)

- Les Systèmes de Gestion de Bases de Données, appelés **SGBD**, ont remplacé les anciens systèmes où les données étaient enregistrées dans des fichiers séparés et liés à chaque application.
- Grâce aux SGBD, toutes les données sont maintenant **rassemblées, organisées** et gérées de manière **centralisée**.

Que fait un SGBD ?

Un SGBD permet de :

- **Décrire** les données (définir les tables, les champs, les relations),
- **Mémoriser** les données,
- **Manipuler** les données avec des requêtes (ajouter, modifier, supprimer, rechercher),
- **Utiliser** les données pour obtenir des résultats utiles

Et il garantit :

- **la sécurité** (protection contre l'accès non autorisé),
- **la confidentialité** (chaque utilisateur a ses droits d'accès),
- **l'intégrité** (les données doivent être justes et cohérentes)

➤ Ce qu'il faut savoir pour utiliser un SGBD



Pour bien utiliser un SGBD, il faut comprendre qu'il permet de :

- **Définir le schéma de données**
c'est la structure de la base (tables, colonnes, types, relations).
- **Faire des opérations sur les données**
recherche, insertion, mise à jour, suppression.
- **Partager les données entre plusieurs utilisateurs**
le SGBD gère les accès simultanés.
- **Optimiser les performances**
le SGBD organise les fichiers internes pour que les accès soient plus rapides.

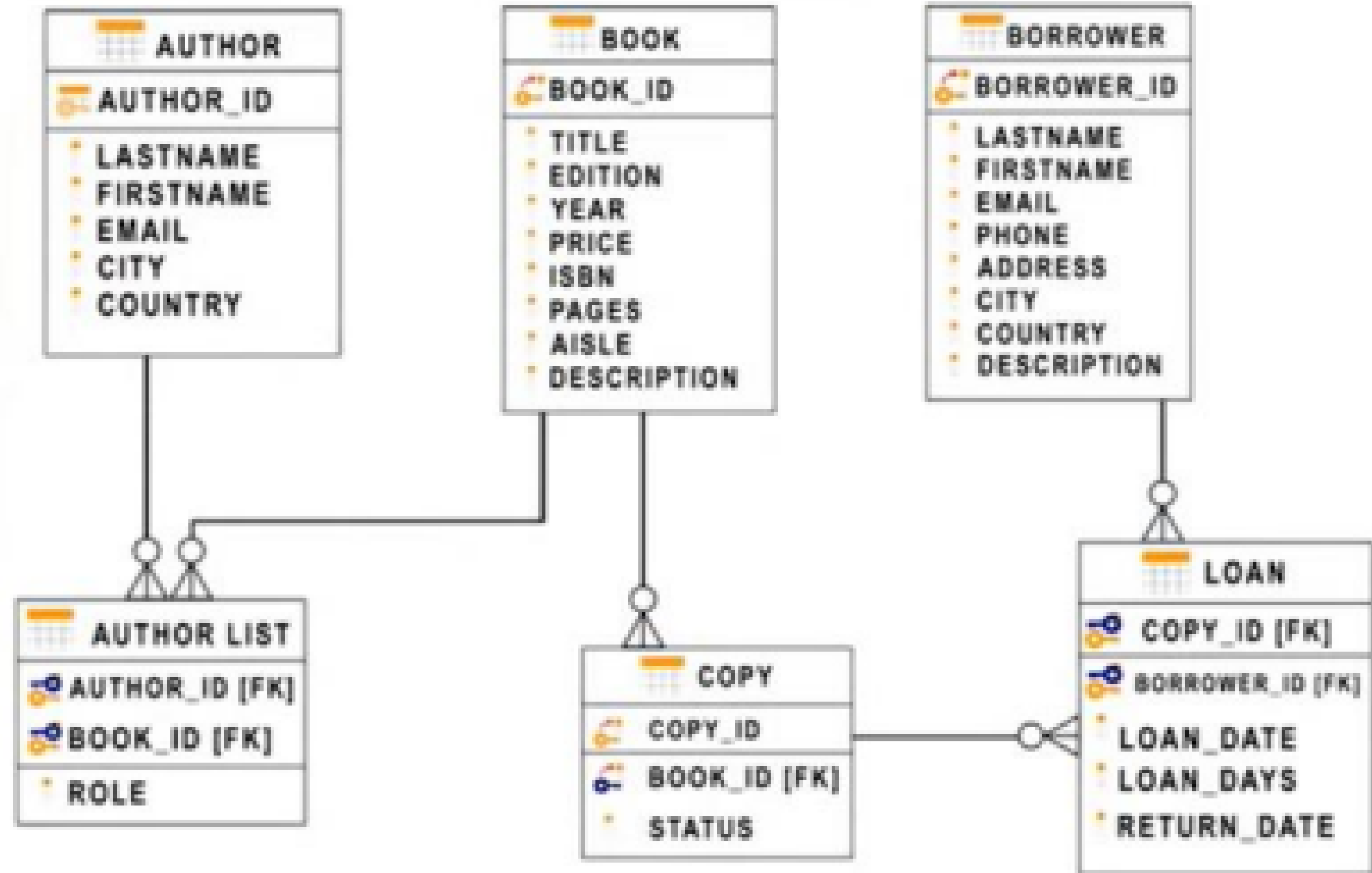




Diagramme Entité-Association



	A	B	C	D	E	F	G
	Titre	Volume	Auteur	Editeur	Prix unitaire	Etat du stock	Total
1	20th Century Boys	1	Naoki Urasawa	Generation Comics	8,99 €	19	180,82 €
2	20th Century Boys	2	Naoki Urasawa	Generation Comics	8,99 €	74	665,26 €
3	20th Century Boys	3	Naoki Urasawa	Generation Comics	8,99 €	49	440,51 €
4	20th Century Boys	4	Naoki Urasawa	Generation Comics	8,99 €	47	421,53 €
5	20th Century Boys	5	Naoki Urasawa	Generation Comics	8,99 €	4	35,96 €
6	20th Century Boys	6	Naoki Urasawa	Generation Comics	8,99 €	47	421,53 €
7	20th Century Boys	7	Naoki Urasawa	Generation Comics	8,99 €	59	529,81 €
8	20th Century Boys	8	Naoki Urasawa	Generation Comics	8,99 €	77	692,23 €
9	20th Century Boys	9	Naoki Urasawa	Generation Comics	8,99 €	46	413,54 €
10	20th Century Boys	10	Naoki Urasawa	Generation Comics	8,99 €	58	521,42 €
11	20th Century Boys	11	Naoki Urasawa	Generation Comics	8,99 €	18	163,82 €
12	20th Century Boys	12	Naoki Urasawa	Generation Comics	8,99 €	55	494,55 €
13	20th Century Boys	13	Naoki Urasawa	Generation Comics	8,99 €	98	881,02 €
14	20th Century Boys	14	Naoki Urasawa	Generation Comics	8,99 €	47	421,53 €
15	20th Century Boys	15	Naoki Urasawa	Generation Comics	8,99 €	58	521,42 €

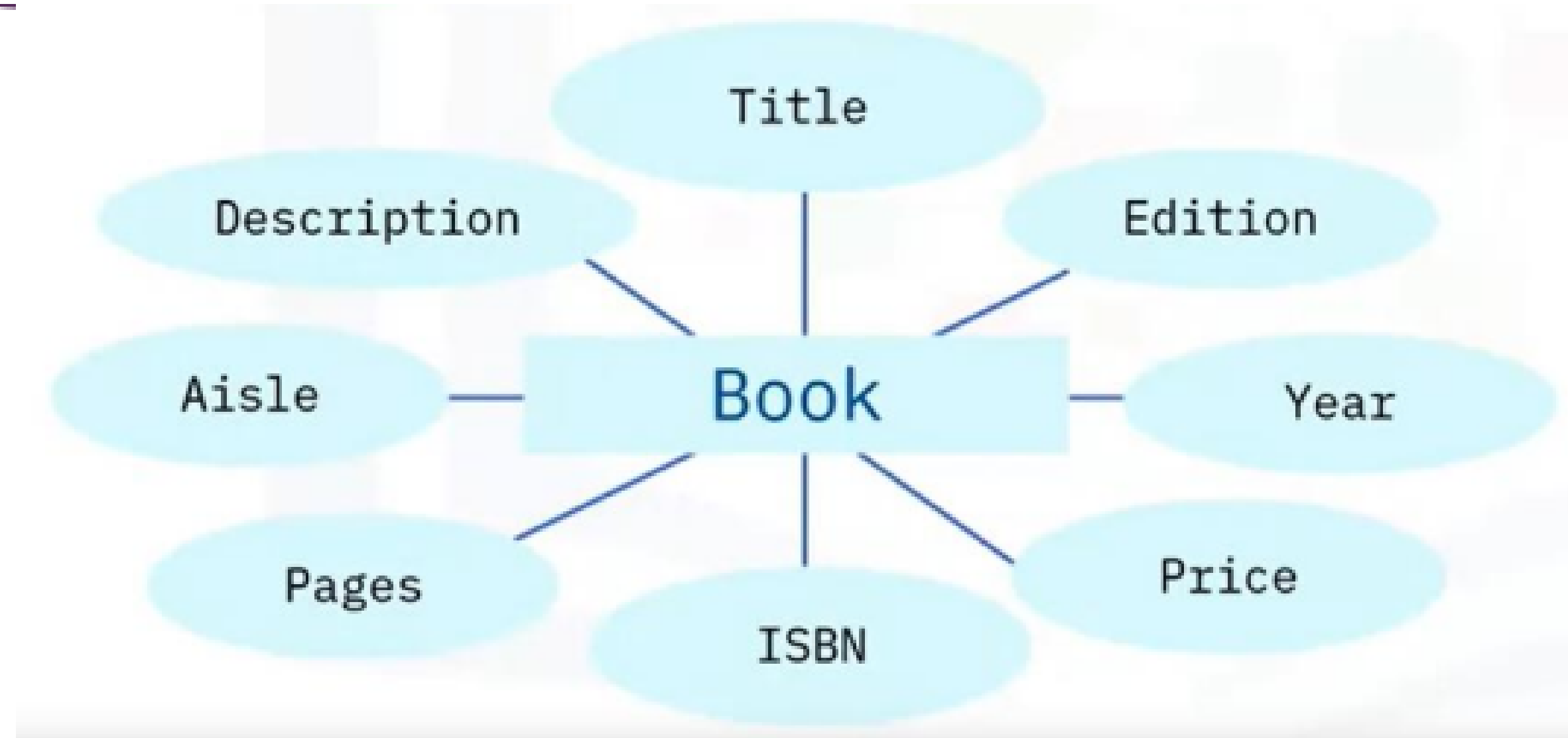


Affichage des données - Employee (Desired.dbg)

	Emp_ID	Last_Name	First_Name	Gender	Title
1	1000	Tobias	Yolanda	F	Programmer
2	1001	Klein	Joel	M	Programmer
3	1002	Ginsburg	Laure	F	President
4	1003	Cox	Jennifer	F	Programmer
5	1005	Zaida	Mauri	M	Product Designer
6	1006	Kayser	Cara	F	Account Executive
7	1010	Smith	Ross	M	Programmer
8	1011	Nelson	Robert	M	Programmer
9	1012	Sachdev	Lam	M	Support Technician
10	1013	Shannen	Don	M	Product Designer

Entité/attribut

BOOK
BOOK_ID
TITLE
EDITION
YEAR
PRICE
ISBN
PAGES
AISLE
DESCRIPTION



Entité: Book

Attributs: Title/edition/description/aisle/pages/isbn/price/year

✧ Cardinalités

Many authors
writing a book

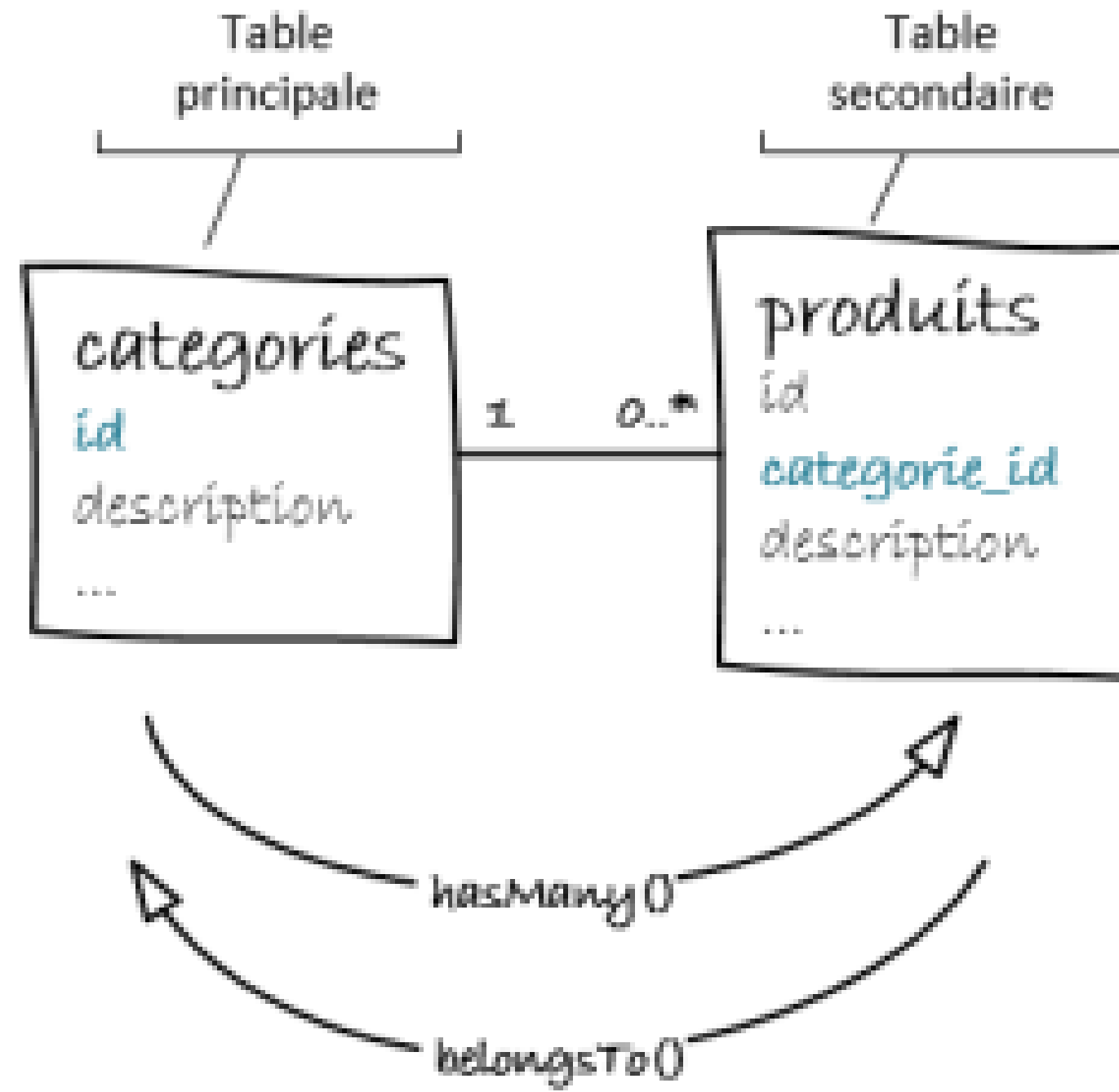


One author writing
multiple books



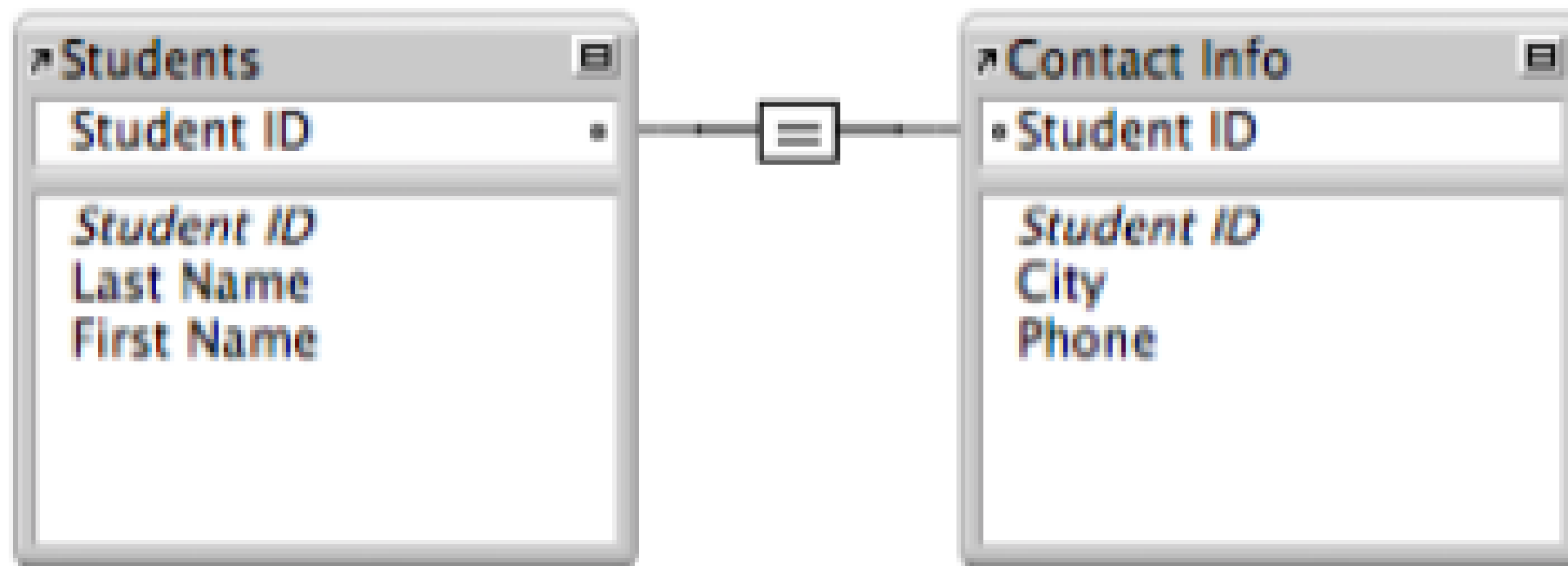
Many-to-Many

Cardinalités



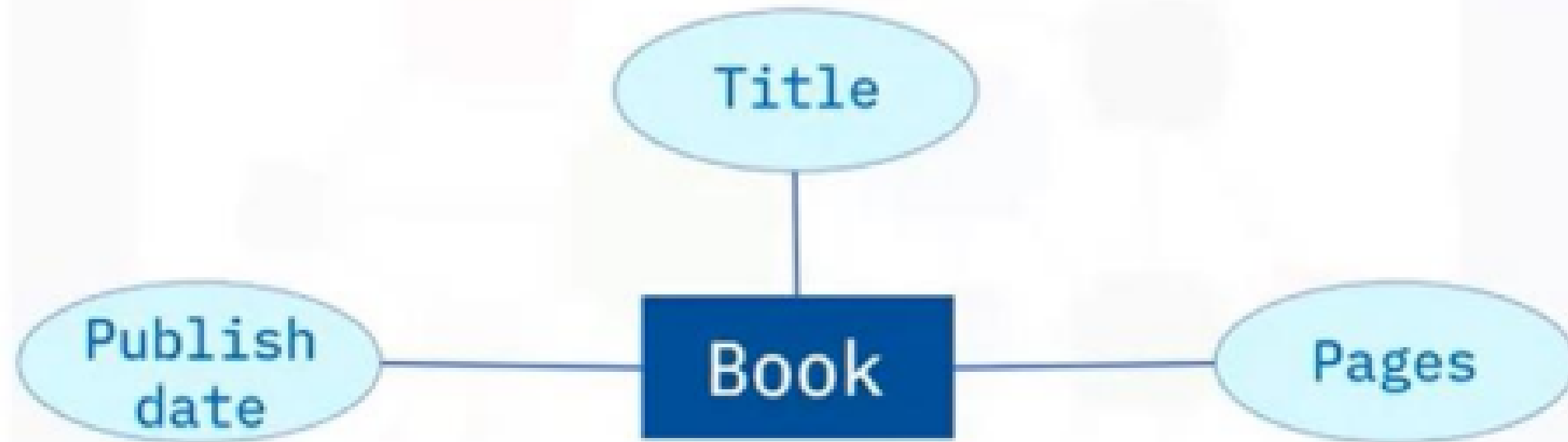
One-to-Many

Cardinalités



One to One

✧ *Type de données* ,



Title	Publish date	Pages
Lean Software Development	2003-05-18	240
SQL Cookbook	2005-12-01	595
Scientific Computing	2001-07-17	576

✧ Type de données

BINARY (Nb_Octet)	Ce type est binaire et à la longueur spécifié en nombre d'octets (groupe de 8 bits).
BIT (Nb_Bit)	Ce type est binaire et à la longueur spécifié en nombre de bits.
BLOB	Ce type est un bloc de données dont la taille maximale est de 2 147 483 647 octets.
CHAR (longueur)	Ce type est une chaîne de caractères ayant une longueur constante maximal de 4000 caractères.
CLOB	Ce type est un texte <i>UNICODE-UCS-2</i> dont la taille maximale est de 2 147 483 647 octets.

DATE	Ce type est un date comprises entre le 1 ^{er} janvier avant Jésus-Christ et le 31 décembre 4712 après Jésus-Christ.
DECIMAL (M,Dec)	Ce type est un nombre ayant longueur texte prédéfini au niveau entier et de ses décimales.
DOUBLE PRECISION	Ce type est un nombre à virgule flottante allant de 10^{-308} à 10^{308} .
FLOAT (p)	Ce type est un nombre à virgule flottante allant de 10^{-308} à 10^{308} .
INTEGER	Ce type est un entier pouvant aller de -2 147 483 648 à 2 147 483 647.

✧ Type de données

NUMBER(<i>M,E</i>)	Ce type est un nombre ayant longueur texte prédéfini au niveau entier entre 1 et 38 chiffres et d'une échelle entre -84 et 127.
RAW(<i>Nb_Octet</i>)	Ce type est un bloc de données dont la taille maximale est de 2 000 octets.
REAL	Ce type est un nombre à virgule flottante allant de 10^{-38} à 10^{38} .
SMALLINT	Ce type est un entier pouvant aller de -32 768 à 32 767.

TIME(<i>Nb</i>)	Ce type est une heure avec le format 24 heures.
TIMESTAMP	Ce type est un date et une heure.
TINYINT	Ce type est un entier pouvant aller de -128 à 127.
VARBINARY(<i>Nb_Octet</i>)	Ce type est un bloc à valeur binaire de taille variable.
VARCHAR(<i>longueur</i>)	Ce type est une chaîne de caractères ayant une longueur variable entre 0 et 4096 caractères.
VARCHAR2(<i>longueur</i>)	Ce type est un une chaîne de caractères ayant une longueur variable entre 0 et 4096 caractères.

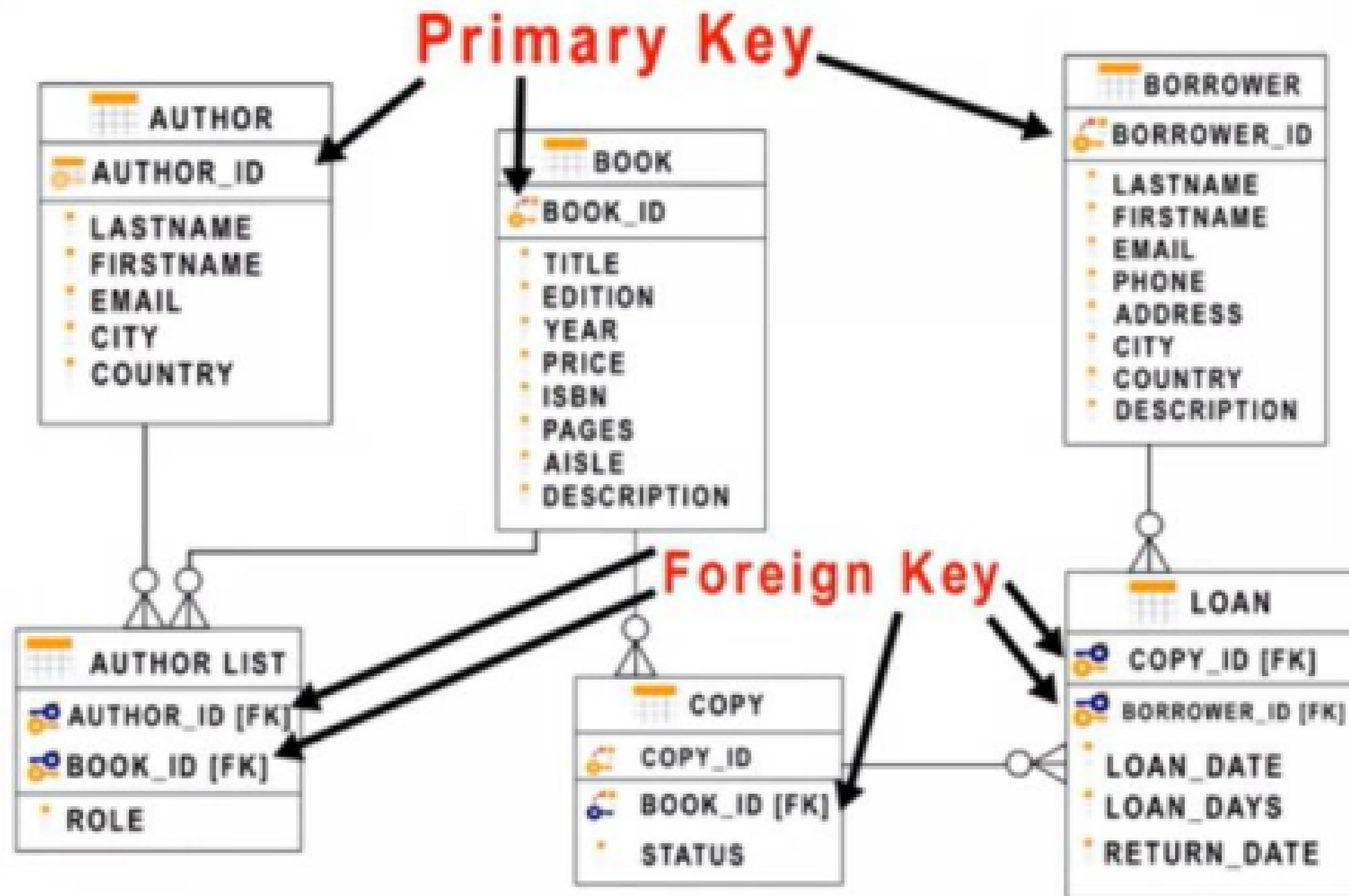
Les contraintes en base de données

Une contrainte est une règle qui permet de garantir la cohérence et la validité des données dans une table.

Types de contraintes :

- **PRIMARY KEY** → identifie chaque ligne de façon unique
- **FOREIGN KEY** → relie deux tables entre elles
- **NOT NULL** → une colonne ne peut pas être vide
- **UNIQUE** → empêche les doublons
- **CHECK** → vérifie une condition

Les contraintes en base de données



✧ Les contraintes en base de données

book_id	title	total_pages	rating	isbn	pub_date	pub_id
101	Lean Software Development	240	4.17	978032	2003-05-18	5
201	Facing the Intelligence Explosion	91	3.87		2013-02-01	109
301	Scala in Action	419	3.74	978194	2013-04-10	111
401	Patterns of Software	256	3.84	978020	1996-08-15	150

copy_id	book_id	location	status
1	101	12	1
2	201	15	0
3	201	8	1
4	101	8	1

Syntaxe de Création des tables

```
CREATE TABLE nom_table (  
    col1 type [(taille)] [DEFAULT valeur_par_defaut] [NULL | NOT NULL] [contrainte],  
    col2 type [(taille)] [DEFAULT valeur_par_defaut] [NULL | NOT NULL] [contrainte],  
    ...  
    [contraintes_de_table]  
);
```

Example

```
CREATE TABLE Book (  
    book_id INT PRIMARY KEY,  
    title VARCHAR(100) NOT NULL,  
    author VARCHAR(100),  
    publish_date DATE,  
    pages INT  
);
```



... On passe à la pratique ! *





Merci !



...

